

实验 3：软件架构与界面设计

24 软工 1 班 24111302044 尹文静

2026 年 9 月

1 实验目的

本次实验要完成三件事：把实验 2 的需求说明转化为可指导开发的架构、模块边界、数据模型、接口与交互方案；形成与个人能力匹配的技术选型并保留决策依据；完成关键设计后检查需求、架构与界面三者是否一致，为实验 4 提供设计基线。

2 实验内容

2.1 需求输入与设计范围

2.1.1 核心用户流程与需求输入

实验 2 为项目定下的是一条 13 环的生产流水线。本次设计从中选定一条核心用户流程作为输入：用户提交一段故事文本，程序调用大模型把它拆成若干漫画分镜，用户在页面上逐格确认并修改，随后逐镜生成图片，最后按顺序浏览整部漫剧。

这条流程覆盖实验 2 用例清单中的七个用例，依次是 UC-01 提交剧情文本、UC-02 生成分镜脚本、UC-03 校验分镜结构、UC-04 确认分镜、UC-05 生成逐格画面、UC-06 局部重跑单格与 UC-11 回放预览作品。实验 2 把这七个用例中的 UC-04 认定为全流程唯一的强制人工确认点，本次设计保留这一判断，并把确认动作放在分镜拆分与图片生成之间。

流程对应的用户故事、非功能需求与验收条件见表 1，编号与措辞均沿用实验 2，不另起一套。

功能需求对应上列七个用例本身，不再单独编号，本次设计即围绕它们展开。

2.1.2 需求与设计对应表（双向追踪）

按双向追踪原则，把每项需求对应到本次要完成的设计内容，结果见表 2。表中前两列在 2.1 节建立，后四列在 2.6 节完成一致性检查后补齐定稿，全表只此一份，2.6 节直接在它上面核对，不另做重复的表格。

2.1.3 普通功能与 AI 功能边界划分

划分的依据是实验 2 定下的架构论点：判断交给 AI，执行交给确定性引擎。落实到这条流程上，四类任务的归属如下。

表 1 设计输入：核心流程对应的需求清单

编号	需求内容	类型	来源
US-01	把写好的剧情文本直接交给系统，不必自己拆镜头、写提示词	用户故事	实验 2 用户故事
US-02	在出图之前先看到 AI 拆好的分镜稿并逐格修改，以便纠正 AI 对剧情的误读	用户故事	实验 2 用户故事
US-04	只重跑不满意的某一格画面，其余部分保持不变	用户故事	实验 2 用户故事
US-08	某一环失败时已生成的结果不丢失并能单独重试	用户故事	实验 2 用户故事
NFR-01	页面操作响应不超过 1 秒；分镜拆分在 60 秒内返回结果	非功能需求（性能）	实验 2 非功能需求
NFR-02	任一环失败不影响已完成环节的产物，每一环支持单独重试，最多自动重试 1 次	非功能需求（可靠性）	实验 2 非功能需求
NFR-03	模型 API 密钥只存放在服务端环境变量中，不写入前端代码与代码仓库	非功能需求（安全）	实验 2 非功能需求
NFR-04	全流程在浏览器内完成；分镜确认页面对每一格都提供可编辑的输入框	非功能需求（可用性）	实验 2 非功能需求
AC-01	提交不少于 100 字的剧情文本后点击“生成分镜”，系统在 60 秒内返回不少于 3 个镜头，每个镜头含序号、画面描述、台词、情绪、建议时长五个字段，序号连续无重复	验收条件	实验 2 验收条件
AC-02	剧情文本为空或不足 100 字时，前端直接拦截并提示最低字数，不发起任何模型调用	验收条件	实验 2 验收条件
AC-03	用户修改某一格的画面描述或台词后点击确认，系统按修改后的内容继续生成	验收条件	实验 2 验收条件
AC-05	某一格图像连续两次生成失败， ² 该格标记为失败并保留其余已完成结果，页面提供重新生成与上一步返回功能	验收条件	实验 2 验收条件

表 2 需求与设计对应表

需求编号	准备完成的设计内容	架构模块	数据或接口	界面	验收条件
US-01	故事提交与分镜拆分接口、分镜 JSON 结构化校验与自动重试	业务层、AI 服务层	POST /api/stories/{id}/storyboards	图 8	AC-01、AC-02
US-02	分镜逐格编辑与保存，改后的内容进入下一环节	界面层、业务层	PATCH /api/storyboards/{id}	图 8	AC-03
US-04	单格图片重新生成，其余分镜的文字与图片不动	AI 服务层、数据层	POST /api/storyboards/{id}/image	图 9	AC-05
US-08	分镜状态字段、失败重试与断点恢复	数据层、业务层	storyboards.status	图 8 错误态	AC-05
NFR-01	前端加载状态与超时提示、接口超时控制	界面层、业务层	接口超时返回 502	图 8 加载态	60 秒内返回
NFR-02	某一格失败不回滚已完成的分镜	数据层	status 字段	—	失败重试不丢数据
NFR-03	.env 环境变量管理，沿用实验 1 已验证方案	全局	.env	—	仓库无明文密钥
NFR-04	分镜卡片内可直接编辑的输入框	界面层	—	图 8	每格可改可存

由确定的程序逻辑完成的是表单校验、故事与分镜入库、图片文件保存与页面渲染，这些任务规则明确，代码直接实现，不需要任何确认。交给 LLM 处理的只有一件事，就是把故事文本拆成带画面描述、台词、情绪与建议时长的结构化分镜，这也是整条流程里唯一没有固定规则可循的环节。需要程序校验的是模型返回的两类结果，分镜 JSON 用它做字段完整性与序号连续性检查，图片则检查文件是否真的生成成功。需要用户确认的是分镜剧情覆盖是否完整、画面与描述是否相符，这两处判断没有客观标准，只能由人过目，也是实验 2 认定的人工确认点所在。

2.2 技术选型与架构决策

2.2.1 候选方案比较

以开发语言与框架为例，按老师点名的五个维度比较三个候选方向，见表 3。其余选型不逐一铺开，只在 ADR 里说明。

表 3 开发语言与框架候选方案比较

比较维度	Python + FastAPI（选用）	Node.js + Express	Java + Spring Boot
熟悉程度	熟悉，实验 1 已完成环境搭建与最小项目	一般	不熟悉
实现成本	低，代码量少，自带接口文档	中	高，配置繁琐
框架生态	Pydantic、OpenAI SDK 齐全	生态丰富	成熟但偏重
AI 接入	OpenAI 兼容 SDK 直接调用	需自行封装 HTTP 调用	需自行封装
测试与运行	uvicorn 本地即起，pytest 可测	需配置较多	需 JVM 环境

2.2.2 设计决策说明（ADR）

ADR-01 开发语言与框架：Python + FastAPI 要解决的问题：选择能够快速实现 Web 接口并方便调用大模型 API 的开发框架。

考虑过的方案：Python + FastAPI、Node.js + Express、Java + Spring Boot。

最终选择：Python + FastAPI。

理由：实验 1 的环境已经就绪，可以直接接着做；Pydantic 能校验大模型返回的 JSON 结构，恰好对上本项目的核心风险——模型输出格式不稳定；框架自动生成的交互式接口文档方便调试。

可能的不足：Python 的并发性能弱于 Node.js，但本项目是单人单机使用，这一差距可以忽略。

ADR-02 数据保存方式：SQLite 要解决的问题：保存故事、分镜与图片路径等结构化数据。

考虑过的方案：JSON 文件、SQLite、MySQL。

最终选择：SQLite。

理由：零部署、单文件即用，能直接表达“一个故事对应多个分镜”的一对多关系；本项目数据量小，单机使用足够。

可能的不足：不支持多用户并发写，本项目为个人软件，可以接受。

ADR-03 体系结构风格：分层架构 要解决的问题：划分界面、业务、数据与 AI 能力之间的模块边界，便于一个人按顺序开发。

考虑过的方案：分层架构、MVC、微服务。

最终选择：分层架构，分为界面层、业务逻辑层、数据访问层与 AI 服务层。

理由：依赖方向单一、边界清晰，一个人可以按层推进；MVC 中视图与控制分离的思路已经体现在页面与接口的划分里。

可能的不足：AI 服务与业务逻辑在同一进程内，若后续模型调用量增大，需要把它拆成独立模块。

ADR-04 图片生成服务：云端文生图 API（火山引擎 Seedream） 要解决的问题：把分镜的画面描述转成漫画风格的图片。

考虑过的方案：本地部署 Stable Diffusion、云端文生图 API。云端这一档又比了火山引擎 Seedream、通义千问 Qwen-Image、即梦与硅基流动 FLUX 四家，其中前三家的计费页面截图见图 1。

最终选择：云端文生图 API，具体用火山引擎的 Seedream。

理由：本机没有独立显卡，本地部署出图慢，云端按张计费不占用本机资源。云端几家的单张价格都在 0.2 元上下，通义千问 0.18 元起、火山引擎 0.2 元，按一条漫剧十来个镜头算约两三元，价格差距小到不构成取舍依据，于是改看另外两条。一条是免费额度够不够跑完课程期，火山引擎新用户有 500 次体验额度，按接口单次出一张图计，本学期的出图量能全部覆盖；即梦走会员积分制，按月订阅换算，成本随订阅档位跳变，估不出单张的真实价格。另一条是计费口径好不好预估，火山引擎的文档写明按调用次数扣费、且只有成功返回图片才计费，成本可以用镜头数直接算出来，硅基流动按每百万像素每一步计费，折算虽然更便宜，但口径不直观，预算不好估。

可能的不足：依赖外部服务的稳定性，画面风格一致性取决于提示词质量，需要保留重新生成能力；500 次额度用尽后转为按量付费，若后续把单集镜头数调大，成本会跟着上去。

2.2.3 关键依赖的替代方案

开发周期跨十几周，关键依赖中途变化的风险不低。风险最高的是 DeepSeek 的分镜拆分，它在整条流程的正中间，一旦不可用后面全部环节都停摆，因此为它预留了明确的退路：DeepSeek 提供 OpenAI 兼容接口，Kimi 与通义千问同样提供兼容接口，切换时只需改动 BASE_URL 与 API 密钥两个环境变量，业务代码一行都不用动。云端文生图服务排在其后，它只影响出图环节，极端情况下可以降级为本地生成的占位图，分镜文字与整条流程仍然可以完整演示。

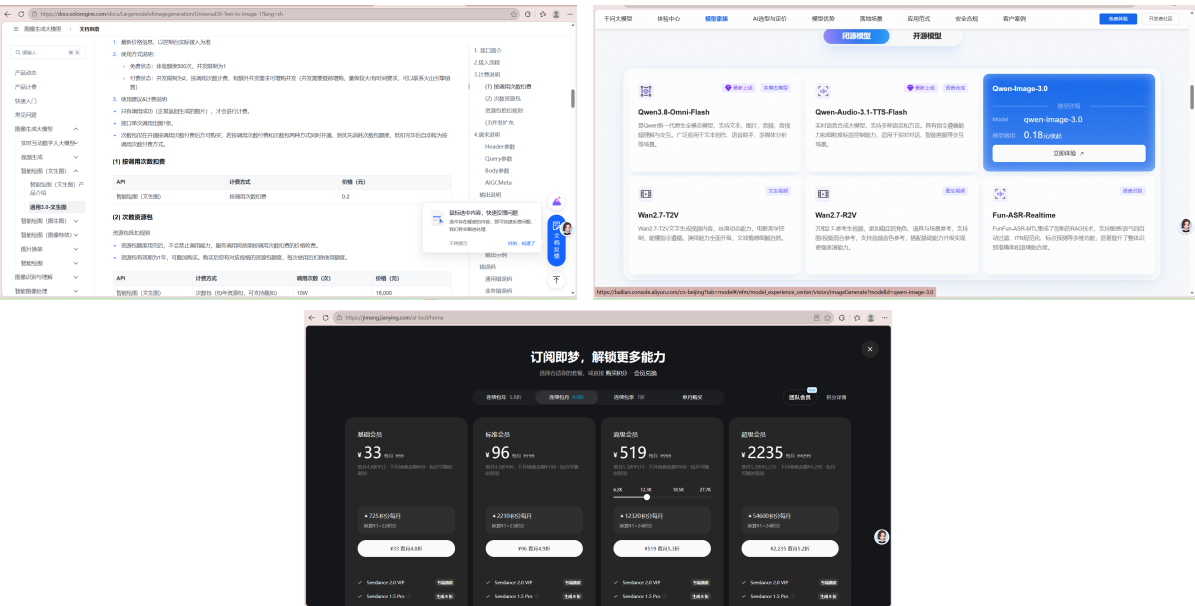


图 1 候选文生图服务的计费页面（自上而下：火山引擎 Seedream、通义千问 Qwen-Image、即梦）

2.3 软件架构与模块设计

2.3.1 软件与外部环境关系图

本次设计范围内的软件与外部环境关系见图 2。

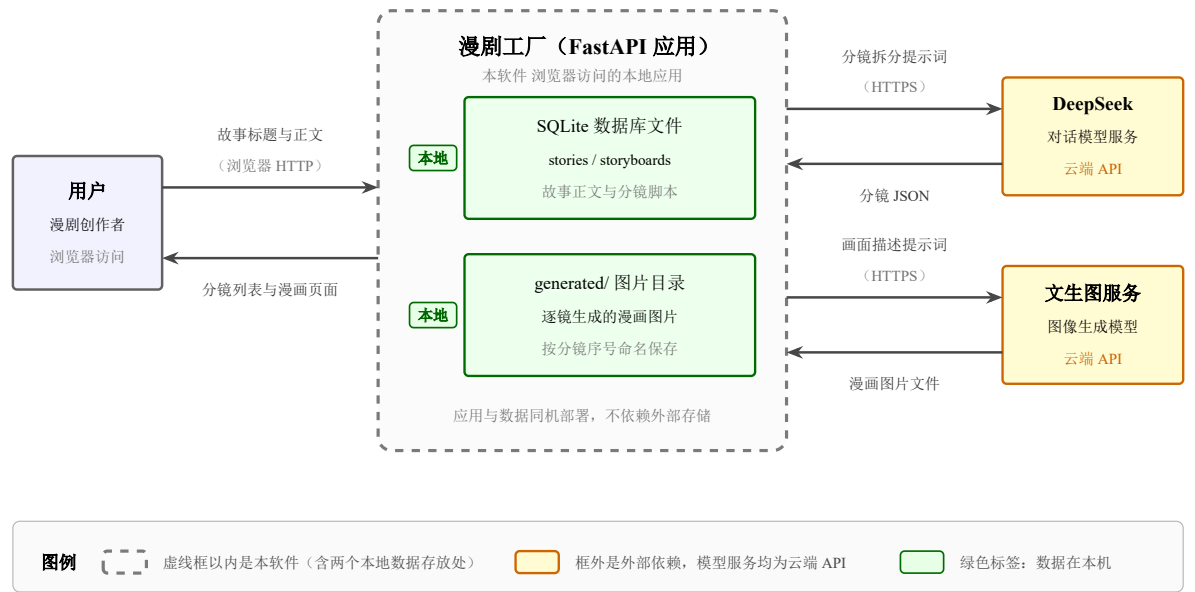


图 2 软件与外部环境关系图

漫剧工厂是一个 FastAPI 应用，连同它自带的 SQLite 数据库文件和 generated/ 图片目录一起构成本软件，三样东西都跑在本机，属于本软件内部。真正的外部依赖只有两个云端模型服务，一个是负责拆分分镜的 DeepSeek 对话模型服务，一个是负责出图的文生图服务，两者都通过 HTTPS 调用。图上流动的数据也标了出来：故事文本由用户经浏览器提交进本软件，组装好的提示词发给 DeepSeek 换回结构化的分镜 JSON，

再按分镜逐条向文生图服务请求图片并把返回的图片文件存进本地图片目录，最后由页面把分镜和图片按顺序渲染给用户。

2.3.2 总体架构图与模块职责

在环境关系图的基础上向内展开，得到软件的总体架构与模块结构，见图 3。

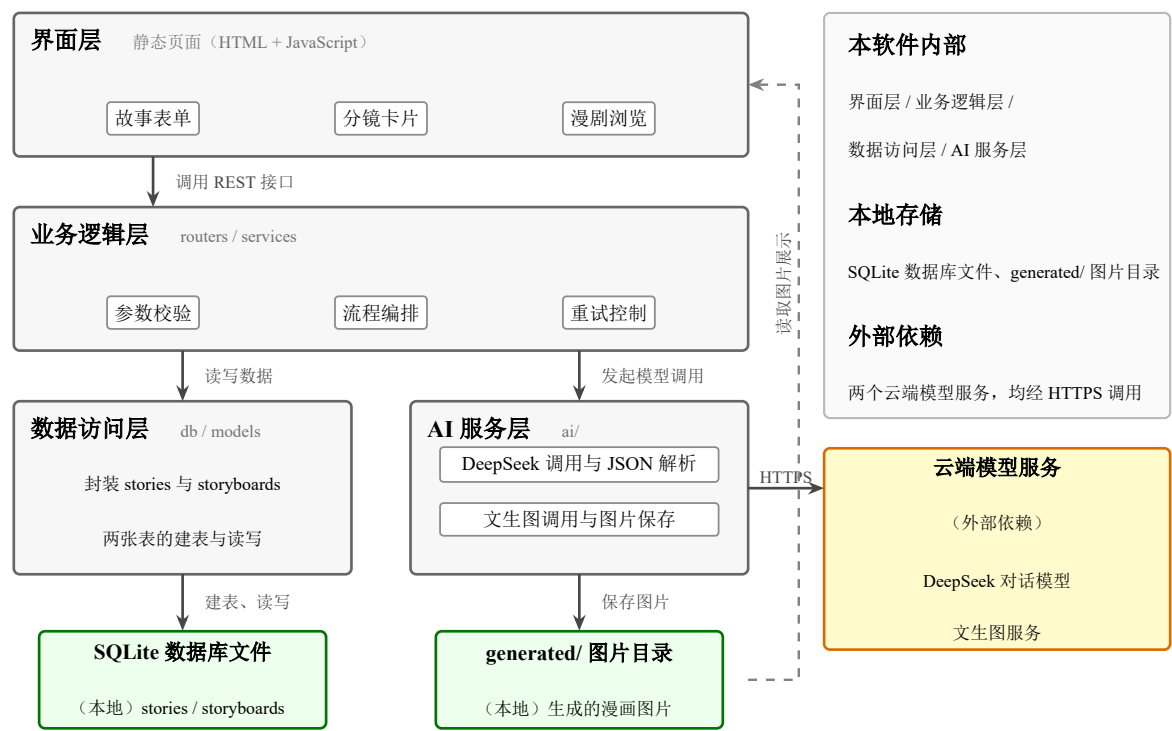


图 3 软件总体架构图

各模块的职责与依赖关系见表 4。界面向下调用业务层，业务层分别调用数据访问层与 AI 服务层，数据访问层读写 SQLite，AI 服务层写 generated/ 目录并经 HTTPS 访问两个云端服务，依赖方向自上而下不回头。分层之后，模型调用全部收拢在 AI 服务层，重试与结构校验只需要在这一层里实现，界面层和数据层都不必感知模型是怎么失败的。

2.3.3 用户—普通程序—模型—工具职责划分表

把核心流程按环节铺开，每一方在各环节里做什么、AI 产出的结果由谁把关，见表 5。

2.4 数据、接口与核心流程设计

2.4.1 核心数据设计

核心数据只有两个对象：一部作品和它的若干分镜。两者的字段、约束与一对多关系见图 4 与表 6。分镜的五个业务字段沿用实验 2 在 UC-02 与 AI 输入输出约定里定下的口径，即画面描述、台词、情绪与建议时长。

表 4 模块职责与依赖关系表

模块	所属层	主要职责	依赖
静态页面（页面目录）	界面层	故事表单提交、分镜卡片渲染、漫剧浏览展示	调用业务层接口
业务逻辑层（routers / services）	业务层	参数校验、流程编排、重试控制、结果返回	数据访问层、AI 服务层
数据访问层（db / models）	数据层	SQLite 建表与读写（stories、storyboards）	SQLite 文件
AI 服务层（ai/）	AI 能力	调用 DeepSeek 拆分分镜并解析 JSON；调用文生图生成图片	外部模型 API
静态资源目录（generated/）	数据层	保存生成的漫画图片，供页面访问	文件系统

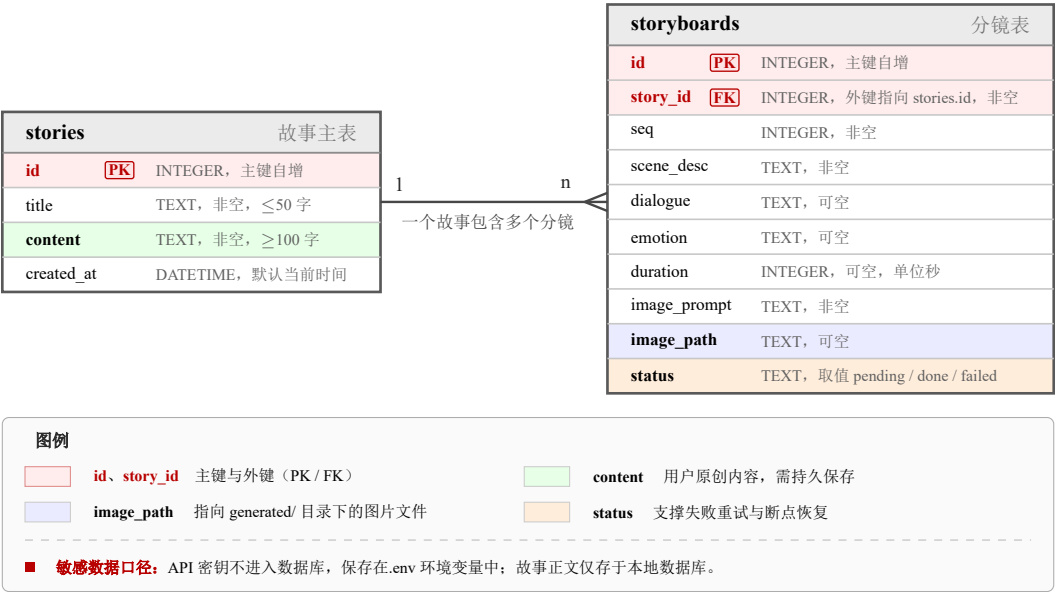


图 4 核心数据 ER 图

敏感数据说明： API 密钥保存在 .env 环境变量中，既不写入数据库，也不暴露给前端，沿用实验 1 已验证的管理方式；故事正文属于用户原创内容，只保存在本地数据库，不用于模型训练。

2.4.2 主要接口设计

支撑这条核心流程的接口共五个，各自的提供模块、调用方、输入输出与失败情况见表 7。

表 5 用户—普通程序—模型—工具职责划分表

环节	用户	普通程序	模型 / 工具	AI 结果检查方
故事输入	填写标题与正文	字数校验、入库	—	—
分镜拆分	提交请求、确认剧情覆盖	组装提示词、调用 API、JSON 校验与重试、入库	DeepSeek 生成含画面描述、台词、情绪与建议时长的分镜 JSON	程序校验字段；用户确认内容
图片生成	确认图文相符、触发重新生成	组装提示词、调用 API、保存文件、记录路径	文生图服务生成图片	程序检查文件；用户确认效果
漫剧浏览	按序浏览	查询分镜并渲染页面	—	—

调用示例：

正常 `POST /api/stories`
 `{ "title": "归乡", "content": "……（100字以上正文）" }`
 返回 200: `{ "story_id": 1 }`

错误 `POST /api/stories`
 `{ "title": "归乡", "content": "" }`
 返回 400: `{ "detail": "故事内容不能为空" }`

表 6 核心数据表设计

字段	类型与约束	说明
stories （故事主表）		
id	INTEGER，主键自增	故事唯一标识
title	TEXT，非空，≤50 字	故事标题
content	TEXT，非空，≥100 字	故事正文（需保存的原创内容）
created_at	DATETIME，默认当前时间	创建时间
storyboards （分镜表）		
id	INTEGER，主键自增	分镜唯一标识
story_id	INTEGER，外键，非空	指向 stories.id，一对多关系
seq	INTEGER，非空	分镜序号，AC-01 要求连续
scene_desc	TEXT，非空	画面描述
dialogue	TEXT，可空	台词
emotion	TEXT，可空	情绪
duration	INTEGER，可空	建议时长（秒）
image_prompt	TEXT，非空	文生图提示词（中间数据）
image_path	TEXT，可空	图片相对路径，位于 generated/
status	TEXT，pending/done/failed	生成状态，支撑失败重试与断点恢复

2.4.3 核心流程

一次分镜生成任务从提交到看到结果，完整走一遍的路径见图 5。

表 7 主要接口设计表

接口	提供模块 / 调用方	用途	输入 / 输出	失败情况
POST /api/stories	业务层 / 首页 表单	创建故事	title、content 进，story_id 出	不足 100 字返回 400
POST /api/stories/{id}/storyboards	业务层、AI 服务层 / 核心任务页	拆分分镜	story_id 进，分镜 JSON 数组 出	超时返回 502；格式错误重试 1 次
PATCH /api/storyboards/{id}	业务层 / 分镜 卡片	保存修改后的分镜文字	storyboard_id、scene_desc、dialogue 进，更新后的分镜出	分镜不存在返回 404
POST /api/storyboards/{id}/image	AI 服务层、业务层 / 分镜 卡片	生成或重新生成单格图片	storyboard_id 进，image_path 出	失败置为 failed，可重生
GET /api/stories/{id}/storyboards	业务层 / 浏览 页	按序获取分镜与图片	story_id 进，分镜列表出	无数据返回空数组

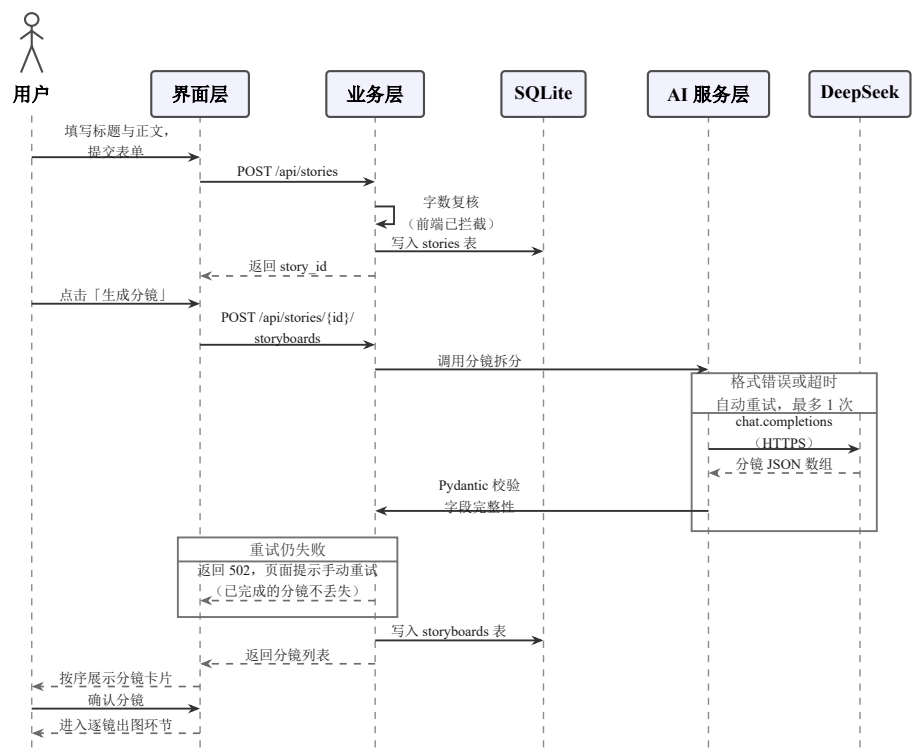


图 5 分镜生成核心流程顺序图

正常路径上，用户提交故事后程序先复核字数并入库，业务层再组装提示词交给 DeepSeek，拿回的分镜 JSON 经 Pydantic 校验通过后写入 storyboards 表，页面按 seq 顺序展示分镜列表。失败处理集中在模型调用这一步，超时或格式错误时自动重试 1 次，仍失败则返回 502，已经生成的分镜不会丢失，由 storyboards 表的 status 字段保证，对应 AC-05。流程中唯一需要人判断的地方在分镜列表展示之后，用户核对剧情覆盖是否完整、必要时直接改分镜文字，确认后再点“生成图片”，这个确认点对应 AC-03，也是整条流程里 AI 结果必须交给给人过目的地方。

2.5 界面原型与交互设计

2.5.1 页面结构与导航关系

软件一共三个页面，彼此的跳转关系见图 6。

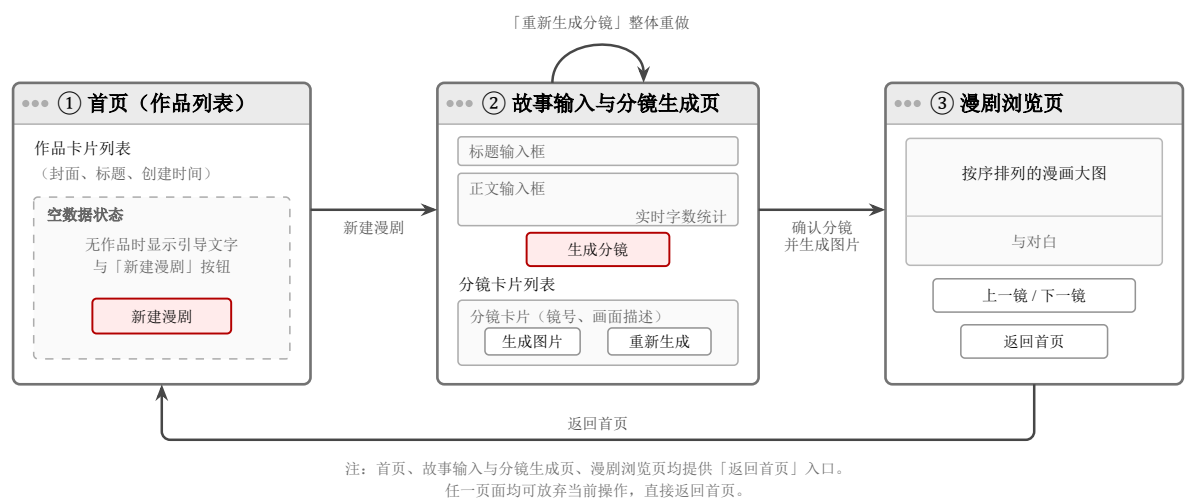


图 6 页面结构与导航关系图

用户从首页进入，点“新建漫剧”打开故事输入与分镜生成页，在那里确认分镜并生成图片，随后进入漫剧浏览页按顺序看完整部作品，最后点“返回首页”退出浏览回到入口。三个页面都能放弃当前操作直接回首页；分镜不满意时，可以在生成页点“重新生成分镜”把整批分镜重做一遍。

2.5.2 关键界面原型图

三张关键页面的原型见下。每张都按四类元素标注：静态元素是页面上不随数据变化的固定信息，包括站点名称、栏目文字与按钮标签；动态元素是随数据和状态变化的区域，包括作品列表、分镜卡片列表、加载动画、实时字数计数器与错误提示；用户输入元素是输入框，首页无需输入，核心任务页有标题与正文两处，标题限制 50 字，正文要求 100 字以上并在右下角实时显示字数；用户命令元素是按钮，包括“新建漫剧”“生成分镜”“生成图片”“重新生成”“返回首页”与浏览页的“上一镜 / 下一镜”，其中“生成分镜”在字数不足时置灰不可点。



图 7 界面原型一：首页

首页是软件入口，展示已经建好的全部漫剧作品。页面上方是站点名称，下方是作品卡片列表，每张卡片显示封面、标题和创建时间，右上角的“新建漫剧”按钮是进入创作流程的唯一入口。作品列表为空时，卡片区域换成一段引导文字和同一个“新建漫剧”按钮，避免新用户停在空白页面上，这也就是老师要求里的空数据状态。



图 8 界面原型二：故事输入与分镜生成页

这一页是整个流程里操作最重的页面，分上下两部分。上半部分是输入区，标题输入框限制 50 字，正文输入框要求 100 字以上并在右下角实时显示字数；正文不满足字数要求时，“生成分镜”按钮置灰，从源头挡住 AC-02 所说的无效请求。下半部分是分镜卡片列表，每张卡片显示序号、画面描述、台词、情绪与建议时长，文字部分直接可编辑，并带一个“生成图片”按钮。

除正常状态外，这一页还要画出四种状态。

- （1）加载状态：提交后按钮变为不可点，卡片区显示加载动画和预计等待时间，对应 NFR-01 提出的 60 秒响应要求。
- （2）错误状态：拆分失败时在卡片区上方显示红色提示文字并给出“重试”按钮，已生成的分镜不消失。
- （3）人工确认状态：分镜文字直接可编辑，用户改完再点“生成图片”，这是全流程唯一的人工确认点。

（4）重新操作状态：单张卡片生成失败时，卡片内显示失败标记和“重新生成”按钮，对应验收条件 AC-05。



图 9 界面原型三：漫剧浏览结果页

浏览页把分镜按 seq 序号纵向排开，每屏一张漫画大图配下方对白，底部是“上一镜 / 下一镜”翻页按钮和“返回首页”。图片还没生成时显示占位框和“去生成”按钮，让用户看得出是缺图而不是加载失败，整页加载过程中显示进度提示。

2.6 设计一致性检查与版本保存

2.6.1 逐项核对结果

按老师要求，检查直接做在 2.1 节建立的表 2 上，把架构模块、数据或接口、界面与验收条件四列补齐，不另做重复的表格，补齐后的结果即为该表。八项需求中有七项在架构、数据、接口、界面四层都找到了落点，NFR-03 属于全局约定，不落在具体模块上，改由仓库检查确认。实验 2 用户故事里的 US-03、US-05、US-06、US-07 分别要求配音合成、导出剪映工程、成片回放与编剧参与确认，都属于流水线后半段，不

在本次选定的核心流程内，因此本次不为它们设计模块与接口，留待后续实验处理。

2.6.2 检查记录与设计调整

检查按两条依据展开：一是每项需求能否在架构、数据、接口、界面四层都找到落点；二是每条验收条件能否在实验 4 直接写成测试用例。查出的问题与对应调整见表 8。

表 8 设计检查记录表

检查范围	检查依据	发现的问题	设计调整
AC-01 的分镜字段约定	分镜字段是否与实验 2 UC-02 及 AI 输入输出约定一致	初稿用了“景别”，与实验 2 约定的“画面描述、台词、情绪、建议时长”对不上，情绪与建议时长两项缺失	表 6 去掉景别，补 emotion 与 duration；图 4 同步重画
AC-03 修改后继续生成	验收条件能否在接口设计上直接找到支撑	初版四个接口只覆盖生成与查询，用户改完分镜没有保存入口，AC-03 无法落地	表 7 增加 PATCH 接口；图 8 的分镜文字改为可编辑
NFR-01 的响应要求	非功能需求在架构与界面上是否都有落点	初稿把分镜拆分的时间限写成 30 秒，与实验 2 AC-01 约定的 60 秒不符；界面上也没有等待反馈	统一按 60 秒表述；接口表补超时返回 502，图 8 补加载状态与等待提示
NFR-03 密钥管理	敏感数据在设计文档中是否有明确落点	架构图与数据设计中都没有出现密钥的存放位置	表 6 后补敏感数据说明，明确密钥走.env，不进入数据库与前端
图 2 外部依赖	老师要求注明本地模型还是云端模型服务	初稿只写了“模型服务”，没有区分本地与云端	两个模型服务统一标注为云端 API，SQLite 与图片目录标注为本地

2.6.3 设计文件版本保存

本次设计的图与报告全部提交到个人仓库，两个远端同步。仓库地址是 <https://github.com/gugujihahaha/AI-ManJv> 与 <https://gitee.com/tianshigirlgj/ai-man-jv>。五张设计图与三张界面原型图放在 docs/design/ 目录下，报告 PDF

放在 docs/report/ 目录下。

图 10 是仓库里 docs/design/ 目录的页面截图，作为老师要求的仓库版本证据。

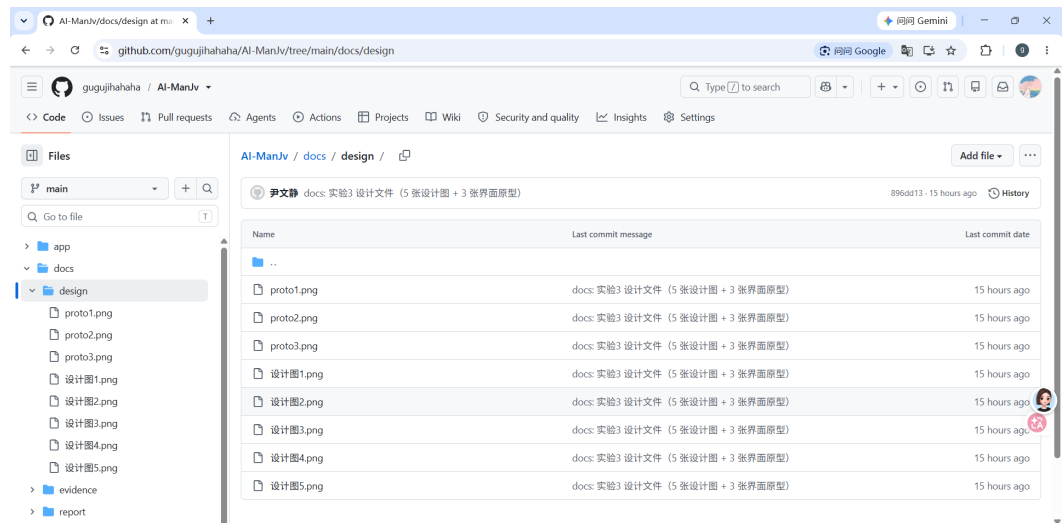


图 10 个人仓库 docs/design/ 目录的页面截图

3 操作记录

本次设计按 2.1 至 2.6 的顺序推进。先选定核心流程并建立需求与设计的对应关系，再依次完成环境关系、总体架构、数据模型、接口定义、核心流程与界面原型，最后回头做一致性检查。

本次设计一共产生十张图。图 1 是候选文生图服务的计费页面截图，作为 ADR-04 选型的依据。图 2 至图 6 是设计图，依次是软件与外部环境关系图、软件总体架构图、核心数据 ER 图、核心流程顺序图与页面结构与导航关系图，与报告一并提交到个人仓库。图 7 至图 9 是界面原型，按老师要求用原型工具或手绘完成。图 10 是仓库页面的截图，用来证明设计文件已经入库。

设计过程中做过两次范围取舍。第一次是把范围从实验 2 的 13 环全景收窄到“故事提交、分镜拆分、逐镜出图、按序浏览”这一条核心流程，范围定小才能把架构、数据、接口与界面四层都落到具体条目上。第二次是数据表只保留 stories 与 storyboards 两张，不为尚未进入本次范围的素材、配音与成片建表，理由是这些环节在本次设计里没有对应的接口去写。

接口设计改过一次。初版把“分镜拆分”与“生成图片”合并成一个接口，按环节铺开后发现两步之间隔着人工确认点，用户在确认前不应该触发图片生成，于是拆成表 7 中的两个独立接口，确认动作落在两次调用之间。一致性检查又发现用户改完分镜没有保存入口，补了 PATCH 接口。

4 实验小结

本次设计把实验 2 的核心流程拆成四层：界面层负责表单提交与页面渲染，业务逻辑层负责参数校验和流程编排，数据访问层封装 SQLite 读写，AI 服务层单独收拢

两个外部模型调用。项目的主要风险是模型输出不稳定，把模型调用集中到一层之后，重试与结构校验只需在这一层实现，界面和存储都不必感知模型的失败方式。选型上，Python + FastAPI 与实验 1 的环境一致，Pydantic 恰好承担分镜 JSON 的字段校验；SQLite 零部署且能表达一对多关系，适合单机个人软件。数据设计保留 stories 与 storyboards 两张表，用 status 字段支撑失败重试与断点恢复，让实验 2 NFR-02 提出的“已完成分镜不丢失”有了明确落点。五个接口全部围绕这条流程，没有为尚未进入本次范围的环节预留接口。

一致性检查以表 2 为主线，把每项需求追到具体的模块、接口、界面与验收条件，共发现五处问题并做了调整，记录在表 8 中。其中影响最大的是分镜字段与实验 2 约定不符、以及缺少保存修改的接口这两条，前者会让 AC-01 的字段校验失据，后者会让 AC-03 无处落地。检查还确认了一件事，AC-05 要求的“失败格标记后保留其余结果”，在把 image_path 与 status 独立成字段之后可以直接写成测试断言。本次形成的架构图、数据模型、接口定义与界面原型将作为实验 4 的开发基线，实验 4 的内容就是把表 7 的五个接口按这里的层次划分实现出来。